❑ Exercice 1 : *Calcul d'une somme*

1. Écrire une fonction `somme` en C de prototype `int somme(int n)` qui renvoie la somme des entiers divisibles par 3 ou par 7 de $[\![1; n]\!]$. Par exemple, `somme(10)` doit renvoyer `25` (`3 + 6 + 7 + 9`).

```c
int somme(int n)
{
    int s = 0;
    for (int i = 1; i <= n; i++)
    {
        if (i % 3 == 0 || i % 7 == 0)
        {
            s += i;
        }
    }
    return s;
}
```

2. Quelle est la valeur renvoyée par `somme(50000)` ?

```
535710714
```

❑ Exercice 2 : *Étendue d'un tableau*

1. Écrire une fonction `etendue` qui prend en argument un tableau (supposé non vide) et sa taille et renvoie l'écart maximal entre deux éléments de ce tableau. Par exemple, sur le tableau `int ex[7] = {1, 5, 3, 0, -1, 4, 8}`, la fonction `etendue` doit renvoyer `9`. *Attention* : on souhaite écrire la fonction `etendue` en utilisant une seule boucle `for`.

```c
int etendue(int tab[], int size)
{
    int mint = tab[0];
    int maxt = tab[0];
    for (int i = 0; i < size; i++)
    {
        if (tab[i] < mint)
        {
            mint = tab[i];
        }
        if (tab[i] > maxt)
        {
            maxt = tab[i];
        }
    }
    return maxt - mint;
}
```

2. Créer un tableau d'entiers `u` de taille 100 et, à l'aide d'une boucle, l'initialiser avec les valeurs prises par la suite $(u_n)_{n\in\mathbb{N}}$ de terme général $u_n = n^2 - 133n + 3822$ pour $n = 0 \dots 99$. C'est-à-dire que `u[i]` doit contenir la valeur de u_i (pour $i \in [\![0; 99]\!]$), par exemple `u[0]=3822`. Déterminer l'écart maximal entre deux éléments du tableau `u` grâce à la fonction `etendue` et donner la réponse trouvée par votre programme :

```c
    int u[100];
    for (int i = 0; i < 100; i++)
    {
        u[i] = i * i - 133 * i + 3822;
    }
    printf("Etendue de u = %d\n", etendue(u, 100));
```

```
4422
```

❑ **Exercice 3** : *Tri par sélection*

On rappelle que le principe du *tri par sélection* est de chercher pour `i=0 ... i=n-1` le minimum du sous-tableau `tab[i] ... tab[n-1]` et de l'échanger avec l'élément d'indice `i`.

Sur le tableau `{2, 8, 1, 5}`, on obtient (en soulignant le minimum de chaque sous-tableau) :

Étapes	État du tableau
Initialisation	{2, 8, 1, 5}
i=0 1er minimum	{1, 8, 2, 5}
i=1 2e minimum	{1, 2, 8, 5}
i=2 3e minimum	{1, 2, 5, 8}

1. Programmer cet algorithme de tri sous la forme d'une fonction de signature :
   `void tri_selection(int tab[], int taille)`
   qui prend en argument un tableau non vide `tab` contenant `taille` éléments et trie en place ce tableau.
   On pourra écrire des fonctions auxiliaires et préciser leur spécification dans le code.

```c
// Renvoie l'indice du minimum des éléments de tab depuis l'indice ind
int min_depuis(int tab[], int taille, int ind)
{
    assert(ind < taille && ind >= 0);
    int vmin = tab[ind];
    int imin = ind;
    for (int i = ind; i < taille; i++)
    {
        if (tab[i] < vmin)
        {
            imin = i;
            vmin = tab[i];
        }
    }
    return imin;
}

// Tri en place un tableau par sélection
void tri_selection(int tab[], int size)
{
    int im;
    for (int i = 0; i < size; i++)
    {
        im = min_depuis(tab, size, i);
        echange(tab, i, im, size);
    }
}
```

2. Créer un tableau `test` de 5000 entiers et initialiser ce tableau de façon à ce que `test[i] = i*i - 564*i + 77760` pour `i=0 ... 4999`. Trier ce tableau à l'aide de la fonction `tri_selection` et donner la valeur de `test[2026]`.

```c
int main()
{
    int test[5000];
    for (int i = 0; i < 5000; i++)
    {
        test[i] = i * i - 564 * i + 77760;
    }
    tri_selection(test, 5000);
    printf("Réponse = %d\n", test[2026]);
}
```

On obtient la valeur 3039772

❑ Exercice 4 : *Nombre harshad*

On dit qu'un entier strictement positif est un *nombre harshad* (ou nombre de Niven) lorsqu'il est divisible par la somme de ses chiffres dans une base donnée. Dans cet exercice, on s'intéresse aux nombres harshad en base 10. Par exemple, 48 est un nombre harshad puisque 48 est divisible par 12, de même 63 (divisible par 9) ou encore 190 (divisible par 10) sont aussi des nombres harshad. Par contre, 28 ou encore 104 ne sont pas des nombres harshad.

1. Écrire une fonction de signature `bool est_harshad(int n)` qui renvoie `true` si et seulement si `n` est un nombre harshad.

```c
bool est_harshad(int n)
{
    int sum = 0;
    int num = n;
    while (num != 0)
    {
        sum += (num % 10);
        num = num / 10;
    }
    return (n % sum == 0);
}
```

2. Écrire une fonction `main` qui prend en argument sur la ligne de commande une chaîne de caractères, la convertit en entier (avec la fonction `atoi`) puis affiche `true` si cet entier est harshad et `false` sinon. Si aucun argument n'est donné, on affiche un message d'erreur. Par exemple, voici des comportements attendus en supposant que l'exécutable s'appelle `harshad.exe` :

```
./harshad.exe 42
true
./harshad.exe 53
false
./harshad.exe
Utilisation : ./harshad.exe <entier positif>
```

```c
int main(int argc, char *argv[])
{
    if (argc != 2)
    {
        printf("Utilisation : %s <entier> \n", argv[0]);
        return EXIT_FAILURE;
    }
    int n = atoi(argv[1]);
    if (est_harshad(n))
    {
        printf("true\n");
    }
    else
    {
        printf("false\n");
    }
    return EXIT_SUCCESS;
}
```

3. Écrire une fonction `int* get_harshad(int limit, int *nb)` qui prend en argument un entier `limit` et renvoie un tableau contenant les nombres harshad inférieurs ou égaux à `limit`. De plus, `*nb` doit contenir après l'appel le nombre de nombres harshad inférieurs ou égaux à `limit`. Par exemple, comme les nombres harshad inférieurs ou égaux à 20 sont `{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 12, 18, 20}`, l'appel `get_harshad(20, &n)` renvoie un tableau contenant ces entiers, et après l'appel, `n` contient la taille de ce tableau, donc `13`.

```c
int *get_harshad(int limit, int *size)
{
    *size = 0;
    for (int i = 1; i <= limit; i++)
    {
        if (est_harshad(i))
        {
            *size = *size + 1;
        }
    }
    int *harshad = malloc(sizeof(int) * (*size));
    int cpt = 0;
    for (int i = 1; i < limit; i++)
    {
        if (est_harshad(i))
        {
            harshad[cpt] = i;
            cpt++;
        }
    }
    return harshad;
}
```

4. On recherche à présent des suites de nombres consécutifs qui sont tous harshad ; par exemple, les nombres 110, 111 et 112 sont 3 nombres harshad consécutifs. Dans un tableau quelconque de valeurs, on est donc amené à chercher le nombre maximal de valeurs consécutives. Écrire une fonction de signature `int consecutifs(int tab[], int size, int *start)` qui renvoie le nombre maximal de valeurs consécutives dans le tableau `tab` de taille `size` et met à jour `*start` afin qu'il contienne la première de ces valeurs. Par exemple, cette fonction sur le tableau `{2, 7, 8, 13, 14, 15, 17, 18, 21}` renvoie 3 et `*start` vaut 13. En effet, il y a un maximum de 3 valeurs consécutives et la première est 13.

```c
int consecutifs(int tab[], int size, int *start)
{
    int cpt = 1;
    int maxc = 1;
    int maxs = 1;
    for (int i = 11; i < size; i++)
    {
        if (tab[i] == tab[i - 1] + 1)
        {
            cpt += 1;
            if (cpt > maxc)
            {
                maxc = cpt;
                maxs = tab[i - cpt + 1];
            }
        }
        else
        {
            cpt = 1;
        }
    }
    *start = maxs;
    return maxc;
}
```

5. Rechercher le nombre maximal de nombres harshad consécutifs parmi ceux qui sont inférieurs à un million *et strictement supérieurs à 9*. Combien y en a-t-il et quel est le premier de cette série ?

   En utilisant les fonctions précédentes, on trouve qu'il y a au maximum 5 nombres harshad consécutifs dans l'intervalle considéré. Le premier d'entre eux est 131 052.